

Universidade do Vale do Itajaí - UNIVALI

M2 - Teoria da Computação

**Trabalho prático 2: Problemas de Roteamento com Análise de Complexidade
e Green Computing - Relatório técnico**

Alunos: Arthur Maia, Gabriel Victor e Gustavo Pedrini

Curso: Ciência da Computação

Professor: Ramicés dos Santos Silva

Data: 20/05/2026

1. Introdução

O presente trabalho aborda uma solução algorítmica para o problema da otimização da ordem de limpeza de vidraças em fachadas de prédios. Esse problema, em um cenário do mundo real, pode ser descrito da seguinte forma: um robô automatizado ou uma equipe de limpeza deve percorrer e limpar todas as janelas sujas em um edifício, dispostas em linhas e colunas. Esse percurso deve ser planejado de modo que os movimentos verticais e horizontais sejam minimizados, assim como o tempo total de operação e o gasto energético do equipamento.

Um dos fatores que dificulta a modelagem deste cenário é o custo de deslocamento variável: movimentos verticais de subida exigem um esforço mecânico maior para elevar a plataforma contra a gravidade, resultando em um maior consumo de energia quando comparado a movimentações horizontais ou de descida. Outro fator de dificuldade seria que, à medida que o número de janelas aumenta, a quantidade de rotas possíveis cresce de forma explosiva, caracterizando este cenário como um problema prático análogo ao Problema do Caixeiro Viajante (PCV) ou ao Caminho Hamiltoniano.

Dito isso, o algoritmo solução deve traçar uma rota que visite cada janela suja exatamente uma vez, seja para retornar à base após o serviço, como no PCV, ou para finalizar na última vidraça (Caminho Hamiltoniano). Nesse cenário, buscar a rota perfeita testando todas as combinações, em uma abordagem de força bruta, por exemplo, torna-se computacionalmente inviável para fachadas maiores, evidenciando a dificuldade de se encontrar a solução ótima para o problema.

Apesar da complexidade, conceber um algoritmo capaz de encontrar uma rota otimizada pode resolver gargalos cruciais da operação no dia a dia. Uma boa solução computacional pode reduzir diretamente o tempo de varredura da fachada, o desgaste mecânico contínuo do robô e também proporciona uma otimização de custos e recursos. Dessa forma, a formulação de rotas eficientes para o equipamento de limpeza traduz-se em uma aplicação prática onde a escolha algorítmica correta afeta não apenas a sustentabilidade do hardware que processa a rota, mas também a eficiência energética da máquina operando no mundo físico, alinhando-se aos princípios de Green Computing.

2. Modelagem do problema

Para a formulação matemática, o cenário físico do prédio foi abstraído e representado através de um grafo modelado em uma grade bidimensional. Nesse grafo, cada janela da fachada que necessita de limpeza é tratada como um vértice. O desafio foi classificado como uma variação do Problema do Caixeiro Viajante, caracterizando-se pela busca de uma rota fechada na qual o equipamento deve visitar todas as janelas sujas exatamente uma vez e, ao final de sua rotina, retornar obrigatoriamente ao seu ponto de partida original, aqui chamado de base. A função de custo definida para avaliar o trajeto entre as janelas não se limita à simples distância geométrica. O custo de deslocamento foi concebido para englobar a distância euclidiana entre as coordenadas acrescida de uma penalidade matemática para movimentos verticais de subida e uma bonificação nos movimentos de descida, refletindo o esforço mecânico extra na subida e a facilidade de descida.

Na investigação das abordagens algorítmicas, duas estratégias principais foram estudadas e implementadas para fins de avaliação e comparação. A primeira consiste em uma abordagem baseada em força bruta através de geração de permutações, que avalia todas as combinações de rotas possíveis para garantir a descoberta matemática da solução ótima. A segunda estratégia foca em uma resolução de tempo factível e utiliza uma abordagem heurística gulosa, especificamente o algoritmo do Vizinho Mais Próximo, que a cada passo da execução seleciona o vértice não visitado de menor custo a partir da posição atual do limpador. Estudos na área de eficiência energética demonstram que mudanças na escolha dos algoritmos, como a substituição de lógicas ineficientes por buscas mais direcionadas, podem reduzir o uso de recursos de processamento a valores próximos de zero em determinadas tarefas.

A implementação computacional destas modelagens foi desenvolvida na linguagem de programação Python, utilizando uma lógica de geração de instâncias aleatórias que simula, por meio de probabilidade, quais janelas estarão de fato sujas a cada nova execução. O programa desenvolvido permite a parametrização do tamanho da malha de janelas diretamente pelo usuário, moldando dinamicamente a topologia do problema em tempo de execução. A estratégia baseada em força bruta apresenta uma complexidade de tempo fatorial, denotada por $O(N!)$, o que inviabiliza sua execução para fachadas contendo um número alto de janelas sujas devido à sobrecarga de processamento. A complexidade de espaço desta estratégia é linear, $O(N)$, relacionada à alocação de memória para as rotas avaliadas. Em contrapartida, o algoritmo do Vizinho Mais Próximo opera com uma complexidade de $O(N^2)$, entregando uma rota com custo relativamente baixo em um tempo muito menor, consumindo também menos ciclos de CPU e demonstrando clara vantagem no escopo de Green Computing. Sua complexidade de espaço também se mantém em $O(N)$, evidenciando um consumo de memória otimizado para o armazenamento das matrizes de visitação.

Algoritmo 1: Força Bruta

```
52 def forca_bruta(janelas: list[tuple[int, int]], callback=None):
53     """
54     Força Bruta O(N!): Testa todas as permutações e retorna à base.
55     callback: função(progresso, total) chamada a cada iteração para mostrar progresso
56     """
57     base = janelas[0]
58     janelas_limpar = janelas[1:]
59
60     melhor_rota = None
61     menor_custo = float('inf')
62     operacoes = 0
63
64     total = math.factorial(len(janelas_limpar))
65
66     for idx, permutacao in enumerate(itertools.permutations(janelas_limpar)):
67         operacoes += 1
68         rota_atual = [base] + list(permutacao) + [base]
69         custo_atual = calcular_custo_rota(rota_atual)
70
71         if custo_atual < menor_custo:
72             menor_custo = custo_atual
73             melhor_rota = rota_atual
74
75         if callback and idx % max(1, total // 100) == 0:
76             callback(idx + 1, total)
77
78     return melhor_rota, menor_custo, operacoes
79
```

O algoritmo de Força Bruta calcula o total de combinações ($N!$) e utiliza o módulo `itertools.permutations` para testar cada rota possível de janelas. Ele calcula o custo de cada ciclo fechado e armazena o menor valor encontrado, garantindo a obtenção da solução ótima global ao custo de um crescimento fatorial de operações.

Algoritmo 2: Vizinho Mais Próximo

```
81 def vizinho_mais_proximo(janelas: list[tuple[int, int]], callback=None):
82     """
83     Vizinho Mais Próximo  $O(N^2)$ : Escolhe a mais próxima e no fim retorna à base.
84     callback: função(progresso, total) chamada a cada passo
85     """
86     base = janelas[0]
87     nao_visitadas = set(janelas[1:])
88     rota_atual = [base]
89     operacoes = 0
90     total = len(janelas)
91
92     while nao_visitadas:
93         janela_atual = rota_atual[-1]
94
95         proxima_janela = min(nao_visitadas, key=lambda j: calcular_custo(janela_atual, j))
96         operacoes += len(nao_visitadas)
97
98         rota_atual.append(proxima_janela)
99         nao_visitadas.remove(proxima_janela)
100
101         if callback:
102             callback(len(rota_atual) - 1, total)
103
104     rota_atual.append(base)
105     custo_total = calcular_custo_rota(rota_atual)
106
107     return rota_atual, custo_total, operacoes
```

Baseado em um heurística gulosa, este algoritmo realiza escolhas locais determinísticas a cada passo, selecionando o vértice não visitado que apresenta o menor custo de transição a partir do ponto atual.

Função de Custo

```
21
22 def calcular_custo(janela1: tuple[int, int], janela2: tuple[int, int]) -> float:
23     """
24     Calcula o custo com:
25     - Descidas (y2 > y1): 0.75x (menos custoso - aproveita gravidade)
26     - Subidas (y2 < y1): 1.5x (mais custoso - gasto energético)
27     - Horizontal: 1.0x (custo normal)
28     """
29     x1, y1 = janela1
30     x2, y2 = janela2
31
32     distancia = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
33
34     if y2 > y1:
35         esforco_vertical = 0.75
36     elif y2 < y1:
37         esforco_vertical = 1.5
38     else:
39         esforco_vertical = 1.0
40
41     return distancia * esforco_vertical
```

Esta função modela matematicamente o impacto energético das transições entre os vértices, calculando a distância euclidiana combinada a um ajuste de peso baseado na direção do movimento. Ao aplicar uma bonificação para trajetos descendentes (0,75x) e uma penalidade para movimentos ascendentes (1,5x), a métrica mimetiza o efeito da gravidade e o esforço mecânico real do equipamento.

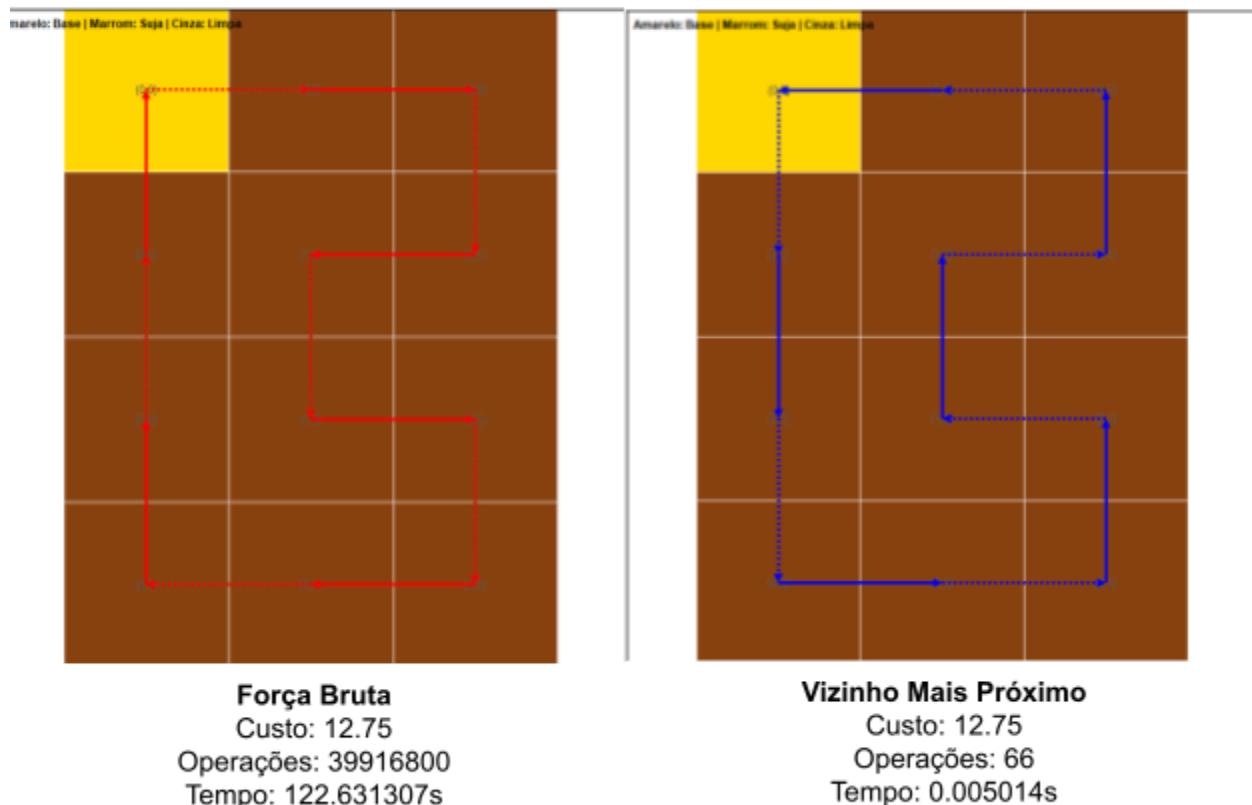
3. Resultados

Para avaliar e comparar o desempenho dos algoritmos de Força Bruta e Vizinho Mais Próximo na determinação das rotas de limpeza, o programa foi submetido a uma bateria de testes projetada para explorar diferentes circunstâncias operacionais. Os cenários buscam simular a variação na escala do problema (tamanho do prédio), na densidade do trabalho (cobertura total versus distribuição aleatória e esparsa) e no impacto do ponto de partida. As simulações contemplaram as seguintes situações:

- Prédio 4x3 com todas as janelas sujas (cenário de cobertura total).
- Prédio pequeno (6x4) com 12 janelas aleatórias sujas (cenário de densidade média).
- Prédio grande (20x10) com 12 janelas aleatórias sujas (cenário de distribuição esparsa).

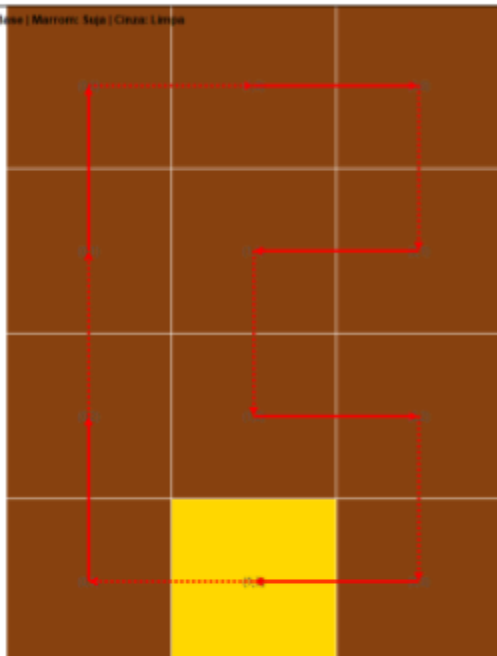
Para cada uma dessas três configurações, realizou-se um teste espelho alterando o início da rota da coordenada padrão (1,1) para um ponto menos conveniente, situado no centro da base do edifício. Essa variação permite analisar como um posicionamento inicial desfavorável afeta a tomada de decisão e a eficiência do caminho traçado por cada algoritmo. Os resultados gráficos de cada simulação, identificados por suas respectivas legendas, são apresentados a seguir.

Prédio 4x3, 12 Janelas sujas, Início (1,1)



Prédio 4x3, 12 Janelas sujas, Início (4,2)

marrom: Base | Marrom: Suja | Cinza: Limpa



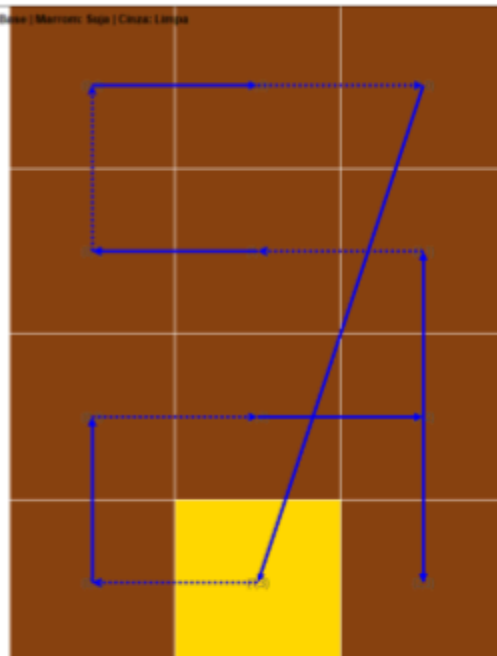
Força Bruta

Custo: 12.75

Operações: 39916800

Tempo: 122.990701s

Amarelo: Base | Marrom: Suja | Cinza: Limpa



Vizinho Mais Próximo

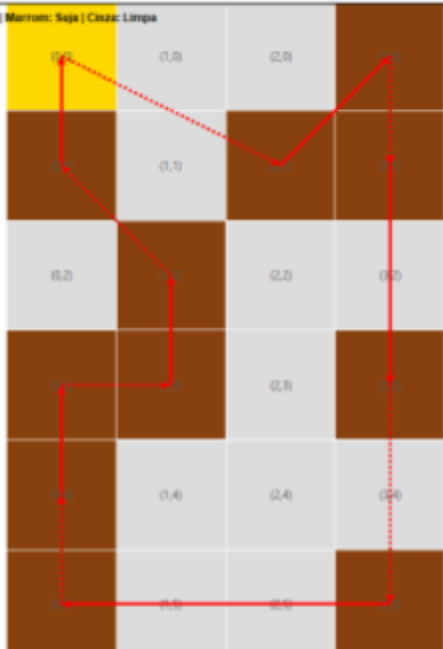
Custo: 16.12

Operações: 66

Tempo: 0.004019s

Prédio 6x4, 12 Janelas sujas, Início (1,1)

marrom: Base | Marrom: Suja | Cinza: Limpa



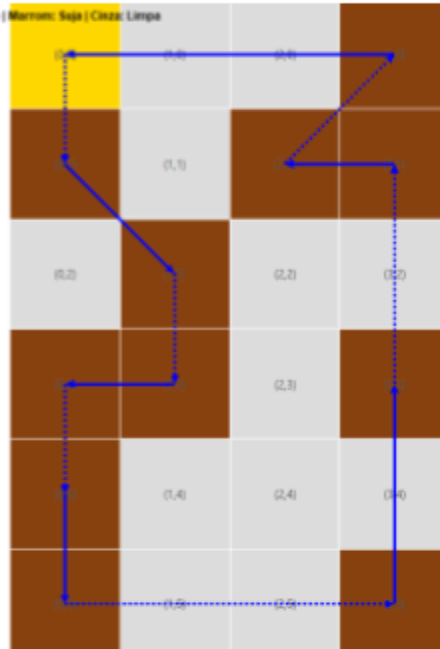
Força Bruta

Custo: 19.67

Operações: 39916800

Tempo: 125.148760s

Amarelo: Base | Marrom: Suja | Cinza: Limpa



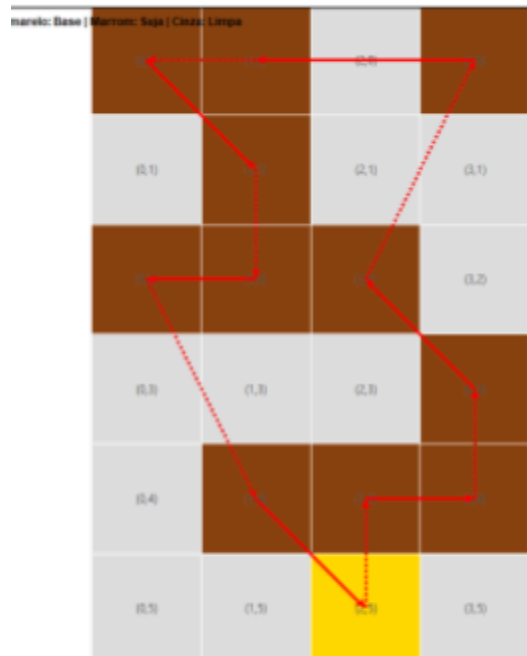
Vizinho Mais Próximo

Custo: 20.18

Operações: 66

Tempo: 0.005516s

Prédio 6x4, 12 Janelas sujas, Início (6,3)

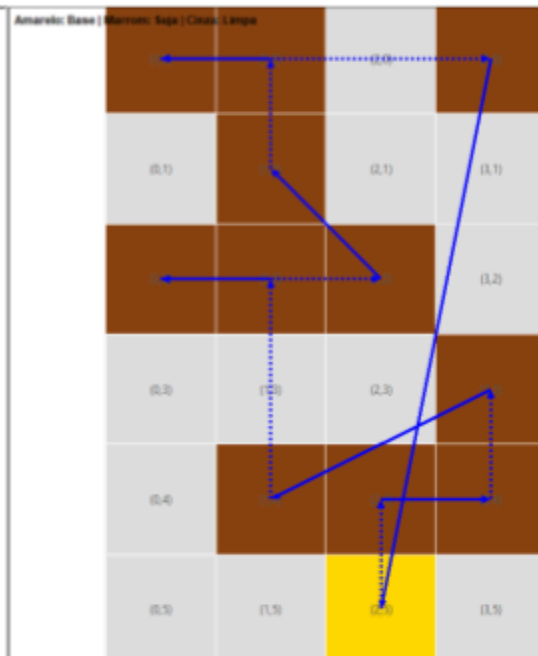


Força Bruta

Custo: 18.02

Operações: 39916800

Tempo: 127.686267s



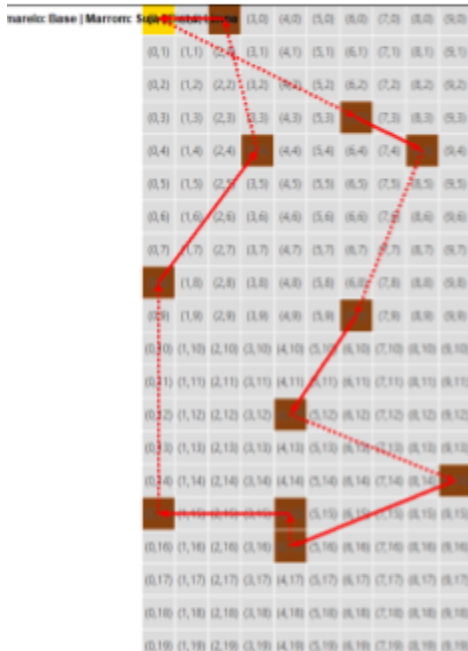
Vizinho Mais Próximo

Custo: 23.12

Operações: 66

Tempo: 0.004013s

Prédio 20x10, 12 Janelas sujas, Início (1,1)

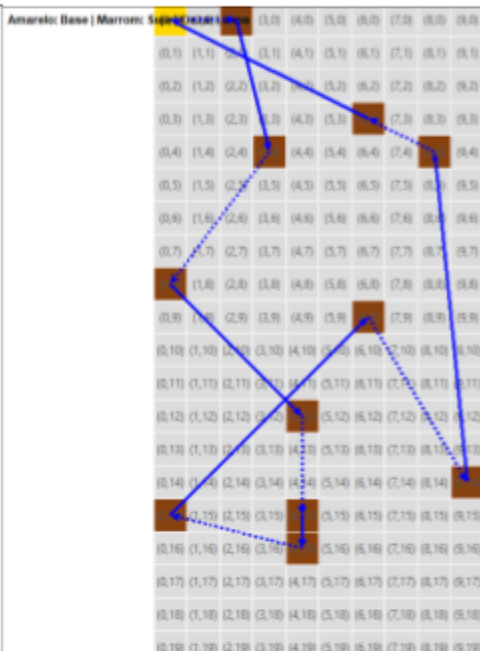


Força Bruta

Custo: 53.21

Operações: 39916800

Tempo: 140.225577s



Vizinho Mais Próximo

Custo: 67.86

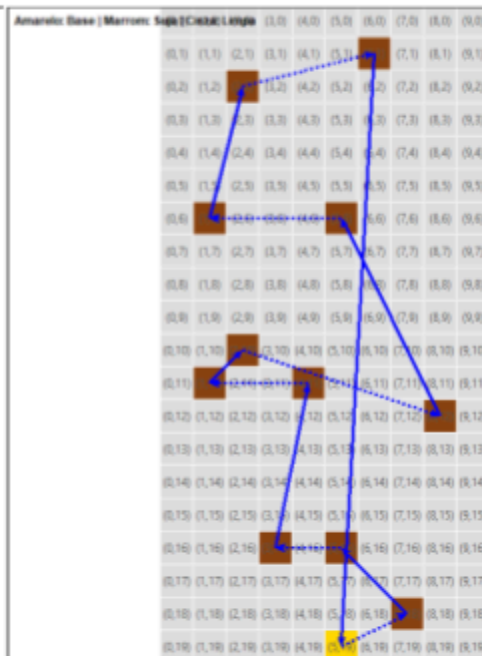
Operações: 66

Tempo: 0.004007s

Prédio 20x10, 12 Janelas sujas, Início (20,6)



Força Bruta
Custo: 51.33
Operações: 39916800
Tempo: 126.559922s



Vizinho Mais Próximo
Custo: 67.06
Operações: 66
Tempo: 0.002506s

4. Análise algorítmica e Green Computing

- **Discussão de Green Computing: síntese dos conceitos principais dos artigos fornecidos:**

A discussão central do estudo consolida a eficácia de arquiteturas algorítmicas específicas na promoção da eficiência energética. O Algoritmo de Redução de Folga (Slack Reduction Algorithm - SRA) demonstra ganhos expressivos de energia em operações densas de álgebra linear, destacando-se nas fatorações de Cholesky e QR.

No campo de alocação de potência, os algoritmos de constelações de bits inteiros apresentam ampla flexibilidade metodológica, podendo ser aplicados a sistemas de constelações não inteiras, desde que fundamentados em conjuntos de pontos discretos e funções que apresentem propriedades de convexidade.

No escopo do agendamento dinâmico de tarefas, o algoritmo duEDF atinge uma economia superior, economizando até 45% de energia quando o foco do gerenciamento é exclusivamente a Unidade Central de Processamento (CPU).

Contudo, ao expandir a análise para o consumo sistêmico global (CPU somada aos dispositivos periféricos), abordagens como duSYS e duSYS PC mostram-se

imperativas, atuando mediante a combinação de configurações ótimas de velocidade com preempção limitada.

Por fim, em ambientes de rede, o algoritmo baseado em Taxa de Computação e Transmissão (CTRB) otimiza o ecossistema ao mitigar a necessidade de o balanceador de carga avaliar de forma contínua o status de consumo elétrico dos servidores subjacentes, reduzindo significativamente a sobrecarga e os gargalos de gerenciamento.

- **Relação entre eficiência algorítmica e consumo de energia no cenário estudado:**

A eficiência de um algoritmo é formalmente caracterizada pela quantidade de recursos computacionais, tempo (complexidade temporal) e capacidade de armazenamento (complexidade espacial), necessários para a sua execução integral.

No cenário investigado, corrobora-se uma relação diretamente proporcional entre o aprimoramento da lógica algorítmica e a conservação energética. O incremento na eficiência resulta na redução substantiva do número de etapas computacionais e na diminuição das exigências de memória.

Essa mitigação no uso contínuo do hardware atenua as dissipações dinâmicas e estáticas de energia.

O impacto ecológico das escolhas de design de software é demonstrável em múltiplas escalas: a substituição de um algoritmo de busca linear de baixa performance por uma arquitetura indexada (como tabelas hash) pode reduzir o uso de recursos para índices próximos a zero.

Em cenários de processamento de dados massivos, otimizações algorítmicas que reduzem o volume de operações de escala quadrática ($N \times N$) para escala logarítmica ($N \times \log(N)$) são capazes de promover economias energéticas análogas aos ganhos obtidos em cinco décadas de avanços físicos em engenharia de hardware.

Dessa forma, a otimização de código atua como vetor essencial da Green Computing.

- **Justificativa de porque a abordagem escolhida pode ser considerada mais ou menos adequada sob a perspectiva de green computing:**

Sob a diretriz da Green Computing, a adequação de um dado algoritmo não é estipulada puramente pelos seus índices nominais de conservação elétrica, mas pela sua complexidade computacional, sobrecarga operacional e aplicabilidade em ambientes produtivos reais.

Essa premissa metodológica é evidenciada no contraste entre os algoritmos de balanceamento de carga EPCLB e CTRB. De um ponto de vista puramente teórico, o algoritmo EPCLB proporciona uma mitigação mais profunda no consumo energético global (redução de 35%) frente ao CTRB (redução de 12%).

Não obstante, o EPCLB é classificado como menos adequado à realidade dos data centers. Seu modelo de seleção exige que o nó balanceador colete o status de consumo (TPCL) de cada servidor em tempo real a cada nova requisição.

Este requisito gera uma carga excessiva de trocas de mensagens na rede, expondo o sistema a atrasos de comunicação e transformando o próprio balanceador no gargalo de processamento da infraestrutura.

Em contrapartida, a implementação do CTRB é justificada como a abordagem mais sustentável e adequada. Ao preterir a análise instantânea do consumo elétrico individual dos servidores, o CTRB simplifica drasticamente as operações do balanceador de carga e minimiza o tráfego interno de mensagens, fornecendo uma economia de energia de 12% de maneira estável, escalável e isenta de gargalos estruturais.

A mesma lógica de adequação pragmática aplica-se ao gerenciamento de hardware: algoritmos abrangentes como o duSYS são metodologicamente preferíveis por considerarem o comportamento energético de todos os periféricos de um sistema, evitando que focos restritos à CPU ignorem a carga estática não-escalável consumida por outros módulos computacionais.

5. Conclusão

Durante os testes, observou-se que o algoritmo do Vizinho Mais Próximo apresenta, no geral, um custo final de deslocamento um pouco superior ao trajeto ideal, uma vez que sua natureza heurística realiza escolhas locais e não garante a descoberta da rota mais curta possível. Entretanto, essa desvantagem na otimização do trajeto é amplamente compensada por uma redução drástica no número de operações executadas. Outro resultado observado foi que testes com a força bruta não podem ser executados quando o número de janelas sujas (vértices do grafo) é maior que 12. Nesses casos, a complexidade fatorial força a geração de muitas combinações de rotas, o que pode exigir horas de processamento da CPU.

Esse esforço computacional massivo para avaliar todas as rotas possíveis acaba violando os critérios estabelecidos pela diretriz de Green Computing. Como a literatura demonstra, a eficiência de um algoritmo está diretamente ligada ao número de passos e ao uso de recursos; rotinas ineficientes desperdiçam capacidade de processamento e, conseqüentemente, aumentam o consumo de energia elétrica. Ao substituir uma busca exaustiva por um algoritmo mais direcionado, é possível reduzir o uso de recursos de forma substancial. Por esse motivo, sob a ótica da sustentabilidade, o algoritmo do Vizinho Mais Próximo é, de longe, o mais recomendado para este problema. Seu número significativamente menor de operações resulta em um tempo de execução muito rápido, garantindo que o hardware opere com eficiência energética máxima e minimize a pegada de carbono do sistema.

Para trabalhos futuros e aprofundamento desta pesquisa, sugere-se a otimização estrutural do código por meio da implementação de processamento paralelo, considerando a distribuição da carga em múltiplas threads. Além disso, será necessário desenvolver tratamentos mais robustos quanto ao limite de iterações máximas da linguagem Python, permitindo que testes mais complexos e pesados rodem sem interromper a execução. Outro ponto crucial de melhoria seria a expansão da função de custo para englobar variáveis físicas que foram abstraídas nesta primeira iteração. A inclusão de parâmetros como as dimensões de cada janela, o peso total da plataforma de elevação e a velocidade variável de movimento mecânico do equipamento tornará a simulação algorítmica ainda mais precisa e fiel aos desafios logísticos do mundo real.